

# Direct Preference Optimization (DPO)

Your Language Model is Secretly a Reward Model

Rafailov et al., NeurIPS 2023

Selim-Antoine Lali, Nazim Fadli, Safa Touzri

APM\_53670 EP Reinforcement Learning  
Master 2 Data Science, Institut Polytechnique de Paris

January 2026



**INSTITUT  
POLYTECHNIQUE  
DE PARIS**



# Outline

- 1 Preference Learning Problem
- 2 Background : RLHF and Its Limitations
- 3 Direct Preference Optimization
- 4 Theoretical Perspective
- 5 Experimental results (IMDb Sentiment Continuation)
- 6 DiffusionDPO : Beyond Language Models
- 7 Conclusion

# Motivation : Why Do We Care ?

- Language models are pre-trained by minimizing the negative log-likelihood :

$$\mathcal{L}_{\text{pre}} = \mathbb{E}_{w \sim \mathcal{D}_{\text{pre}}} \left[ - \sum_{t=1}^{T_w} \log \pi_{\ell}(w_t \mid w_{<t}) \right]$$

- This objective optimizes **likelihood**, not **human preference** :

high likelihood  $\not\Rightarrow$  desirable behavior

- Many desired properties are difficult to encode as scalar rewards :
  - Helpfulness, safety, alignment
  - Subjective and context-dependent
- Humans can instead provide **pairwise preferences** :

$$y^w \succ y^l \quad \text{for a given prompt } x$$

# Problem Setting

- We are given a dataset of **human preferences** :

$$\mathcal{D} = \{(x, y^w, y^l)\}$$

where :

- $x$  is a prompt (state)
  - $y^w$  is the preferred response (action)
  - $y^l$  is the less preferred response (action)
- We consider a parameterized policy :

$$\pi_{\theta}(y \mid x)$$

- Goal : learn a policy that assigns higher probability to preferred responses :

$$\pi_{\theta}(y^w \mid x) > \pi_{\theta}(y^l \mid x)$$

- RLHF aligns a pretrained language model with human preferences
- Training proceeds in three stages :

❶ **Supervised Fine-Tuning (SFT)** on demonstrations :

$$\mathcal{L}_{\text{SFT}} = \mathbb{E}_{(x,y) \sim \mathcal{D}_{\text{SFT}}} \left[ - \sum_{t=1}^{T_y} \log \pi_{\ell}(y_t \mid x, y_{<t}) \right]$$

❷ **Reward Modeling** : learn a scalar reward

$$r_{\phi}(x, y) \quad \text{from preference pairs } (y^w, y^l)$$

❸ **RL Fine-Tuning** with PPO :

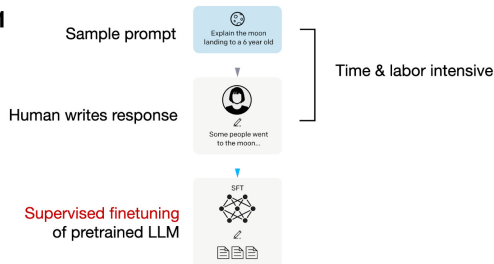
$$\max_{\pi_{\theta}} \mathbb{E}_{y \sim \pi_{\theta}(\cdot|x)} \left[ r_{\phi}(x, y) - \beta \text{KL}(\pi_{\theta} \parallel \pi_{\text{ref}}) \right]$$

# RLHF Step 1 : Supervised Fine-Tuning

- Humans write high-quality responses
- Fine-tune pretrained LM with MLE
- Produces initial policy :

$$\pi_{\text{SFT}} \approx \arg \min_{\pi} \mathcal{L}_{\text{SFT}}$$

## RLHF Step 1

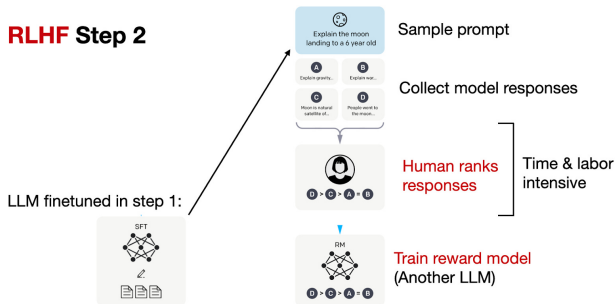


Figures adapted from *InstructGPT* (Ouyang et al., 2022), arXiv:2203.02155

# RLHF Step 2 : Learning a Reward Model

- Sample responses from  $\pi_{\text{SFT}}$
- Humans rank responses
- Train reward model  $r_{\phi}(x, y)$  via preference likelihood maximization

## RLHF Step 2



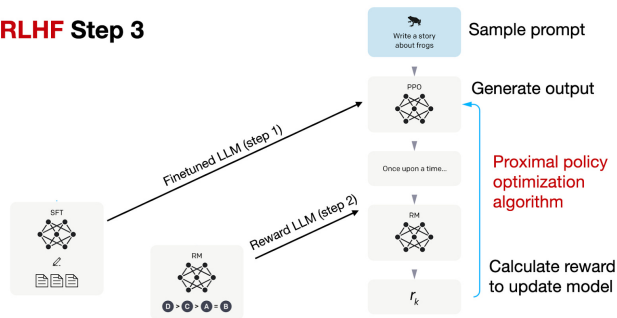
$$\mathbb{P}(y^w \succ y^l \mid x) = \sigma(r_{\phi}(x, y^w) - r_{\phi}(x, y^l))$$

Figures adapted from *InstructGPT* (Ouyang et al., 2022), arXiv:2203.02155

# RLHF Step 3 : RL Fine-Tuning with PPO

- Policy samples responses
- Reward model evaluates them
- PPO updates policy with KL constraint

## RLHF Step 3



$$\max_{\pi_{\theta}} \mathbb{E}_{y \sim \pi_{\theta}} [r_{\phi}(x, y)] - \beta \text{KL}(\pi_{\theta} \parallel \pi_{\text{ref}})$$

Figures adapted from *InstructGPT* (Ouyang et al., 2022), arXiv:2203.02155



# Limitations of RLHF

- **Two-stage optimization :**
  - Learn a reward model  $r_\phi(x, y)$  from preferences
  - Optimize policy  $\pi_\theta$  against  $r_\phi$  using RL
- **Instability of on-policy RL :**
  - PPO requires on-policy sampling
  - Sensitive to KL coefficients and clipping
  - Susceptible to reward hacking and mode collapse
- **High computational and engineering cost :**
  - Sampling inside the training loop
  - Multiple large models trained and tuned jointly
- **Limited scalability and reproducibility in practice**

- Learn a policy *directly* from preference data
- No explicit reward model, no RL loop
- Optimization is relative to a reference policy  $\pi_{\text{ref}}$

**Key objective :** make preferred responses more likely than rejected ones

$$\pi_{\theta}(y^w | x) > \pi_{\theta}(y^l | x)$$

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x, y^w, y^l) \sim \mathcal{D}} \left[ \log \sigma \left( \underbrace{\beta \left[ \log \frac{\pi_{\theta}(y^w | x)}{\pi_{\text{ref}}(y^w | x)} - \log \frac{\pi_{\theta}(y^l | x)}{\pi_{\text{ref}}(y^l | x)} \right]}_{\text{relative preference vs reference}} \right) \right]$$

**Intuition :**

- Increase likelihood of preferred response  $y^w$
- Decrease likelihood of rejected response  $y^l$
- Penalize deviations from  $\pi_{\text{ref}}$  implicitly

# From RLHF to DPO (closed-form policy)

- Standard RLHF optimizes a KL-regularized reward objective :

$$\max_{\pi} \mathbb{E}_{y \sim \pi(\cdot|x)} \left[ r(x, y) - \beta \text{KL}(\pi(\cdot|x) \parallel \pi_{\text{ref}}(\cdot|x)) \right]$$

- Main theoretical insight : this objective admits a **closed-form optimal policy**

$$\pi^*(y|x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y|x) \exp\left(\frac{1}{\beta} r(x, y)\right)$$

- where  $Z(x) = \sum_y \pi_{\text{ref}}(y | x) \exp\left(\frac{1}{\beta} r(x, y)\right)$ , Partition function ensuring probabilities sum to 1.
- The optimal policy is a **reweighted reference policy**
- Higher-reward responses receive exponentially higher probability

# Reward reparameterization

- Closed-form optimal policy :

$$\pi^*(y|x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y|x) \exp\left(\frac{1}{\beta} r(x, y)\right)$$

- Taking logs gives the reward representation :

$$r(x, y) = \beta \log \frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} + \beta \log Z(x)$$

- Additive constant  $\log Z(x)$  does **not depend on  $y$  (action)**
- Preferences compare **two responses**

$$r(x, y^w) - r(x, y^l) = \beta \left[ \log \frac{\pi^*(y^w|x)}{\pi_{\text{ref}}(y^w|x)} - \log \frac{\pi^*(y^l|x)}{\pi_{\text{ref}}(y^l|x)} \right]$$

# Preferences to likelihood

- Human data provides **pairwise preferences** :

$$(x, y^w, y^l) \quad \text{with } y^w \succ y^l$$

- Preferences are modeled using a **Bradley–Terry** likelihood :

$$\mathbb{P}(y^w \succ y^l \mid x) = \sigma(r(x, y^w) - r(x, y^l))$$

- Using the previous result, reward differences depend only on the policy
- The normalization constant cancels out

**Key takeaway** : Preferences define a likelihood directly over policy parameters.

# DPO as Maximum Likelihood Estimation

- Preference data induces a likelihood over policy parameters
- Training reduces to optimizing a standard logistic objective

$$\min_{\theta} \mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x, y^w, y^l) \sim \mathcal{D}} \left[ \log \sigma \left( \beta \left( \log \frac{\pi_{\theta}(y^w|x)}{\pi_{\text{ref}}(y^w|x)} - \log \frac{\pi_{\theta}(y^l|x)}{\pi_{\text{ref}}(y^l|x)} \right) \right) \right]$$

## Key takeaway :

- DPO maximizes the likelihood of observed preferences (recovers optimal policy  $\pi^*$  directly via likelihood maximization).
- DPO replaces unstable RL with a standard objective while preserving the optimal policy.
- In DPO, we never train an explicit reward model  $r_{\phi}(x, y)$
- Yet, DPO behaves *as if* a reward model exists

# Your Language Model Is Secretly a Reward Model

$$\nabla_{\theta} \mathcal{L}_{\text{DPO}}(\pi_{\theta}; \pi_{\text{ref}}) = -\beta \mathbb{E}_{(x, y^w, y^l) \sim \mathcal{D}} \left[ \underbrace{\sigma(\hat{r}_{\theta}(x, y^l) - \hat{r}_{\theta}(x, y^w))}_{\text{higher weight when reward estimate is wrong}} \underbrace{\nabla_{\theta} \log \pi_{\theta}(y^w | x)}_{\text{increase likelihood of } y^w} - \underbrace{\nabla_{\theta} \log \pi_{\theta}(y^l | x)}_{\text{decrease likelihood of } y^l} \right]$$

**Implicit reward definition (from theory) :**

$$\hat{r}_{\theta}(x, y) := \beta \log \frac{\pi_{\theta}(y | x)}{\pi_{\text{ref}}(y | x)}$$

- $\hat{r}_{\theta}(x, y)$  acts as an **implicit reward function**
- It is entirely defined by the language model itself
- Relative to a fixed reference policy  $\pi_{\text{ref}}$

**Interpretation :**

- Higher likelihood than reference  $\Rightarrow$  higher reward

# Experimental Setup

## Task : IMDb controlled sentiment continuation

- **Input** : prompt  $x$  : first  $W=8$  words of IMDb review
- **Output** : continuation  $y$  of up to  $T = 64$  new tokens
- **Goal** : maximize *positive sentiment* while staying close to a reference LLM

## Models & preference data

- **Base/reference policy** : GPT-2 Large (frozen  $\pi_{\text{ref}}$ )
- **Reward model** : RoBERTa reward  
siebert/sentiment-roberta-large-english
- **Offline prefs** : for each prompt, sample  $k = 4$  candidates, build **all pairs** (6) as preference pairs
- **Fine-tuning** : LoRA adapters (train  $\approx 1\%$  of params)

## Evaluation metrics

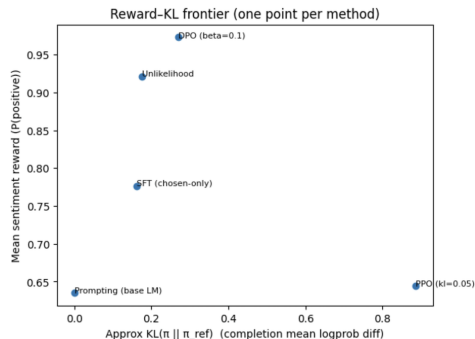
- **Mean reward** :  $\mathbb{E}[r(y)]$ ,  $r(y) \in [0, 1]$
- **Positive rate** :  $(r(y) > 0.5)$
- **Approx “KL” divergence** :  
$$\Delta \log p = \mathbb{E}[\log \pi_{\theta}(y|x) - \log \pi_{\text{ref}}(y|x)]$$
- mean per generated token, computed on completion tokens.

## Compared baselines

- Prompting (base LM), Best-of- $N$
- SFT (chosen-only), Unlikelihood
- DPO (sweep over  $\beta$ )
- PPO (sweep over KL coef)



# Results : Reward-KL Frontier (IMDb)



Run 1 : one evaluation point per method.

- Base LM : facebook/opt-350m
- Reward model : lvwerra/distilbert-imdb
- Prompt length :  $W = 35$  words
- Preferences :  $k = 4$  candidates/prompt, keep only *best vs worst* pair

## Key observations

- **DPO** reaches the highest rewards at relatively small drift from  $\pi_{ref}$
- **Unlikelihood** achieves large reward improvements at modest divergence.
- **PPO** is the hardest to tune : more instability and weaker frontier in our setup
- **Best-of- $N$**  achieves very high reward, but at **high inference cost**

## Evaluation metrics

| Method              | Mean reward | Pos. rate | Approx KL |
|---------------------|-------------|-----------|-----------|
| Best-of-64          | 0.993       | 1.000     | –         |
| Prompting (base)    | 0.635       | 0.643     | 0.000     |
| SFT (chosen-only)   | 0.776       | 0.798     | 0.161     |
| Unlikelihood        | 0.921       | 0.948     | 0.175     |
| DPO ( $\beta=0.1$ ) | 0.973       | 1.000     | 0.270     |
| PPO ( $kl=0.05$ )   | 0.645       | 0.703     | 0.887     |

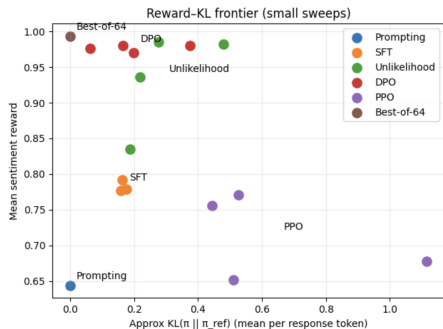
# Qualitative generations (reward hacking)

One prompt, one sampled continuation per method (Run 1)

| Item                 | Text (output)                                                                                                                                                                             |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| IMDb prompt $\times$ | Probably the best comedy in a long time. keeps you laughing nonstop! the acting is good and there are a lot of hilarious cameos such as Ben stiller as the guitar store guy. The plot     |
| Base (0.490)         | just needs to come together and end. Edit : words. Yeah, the plot is already a thing. But the characters in this show seem like the most interesting, intelligent, and funny people in... |
| SFT (0.992)          | is really funny and very much in keeping with the style of the show. I can't wait to see what the next season brings! I was honestly hoping for that ending. The comedy was so good...    |
| Unlikelihood (0.995) | is very well put together and I am interested in a lot of the characters who are involved, such as the wife and their son, as well as the characters that are involved in the show. I...  |
| DPO (0.973)          | great great great great great great great great great great great ...                                                                                                                     |
| PPO (0.734)          | "the and. on a you the or, of will and which ". in are the and the in or . to and or the of i, is the or in and it and; to; ( a or or a the -; i i of or in and and ( f and               |

**Takeaway :** Optimizing a sentiment classifier can induce **reward hacking** (e.g., repetitive triggers like “great great ...”). Some methods (notably PPO) can also degrade fluency (token soup / collapse).

# Results : Reward-KL Frontier (IMDb)



Run 2 : small sweeps over each method's hyperparameter.

- Same experimental setups as Run 1.

## Key observations

- Sweeping the knob produces a **frontier curve**.
- **DPO** and **Unlikelihood** achieve **high reward** with **moderate KL drift**.
- **SFT** improves reward but remains clearly below DPO/UL.
- **PPO** is highly sensitive : reward gains come with **large KL** and less stable behavior.
- **Best-of-64** reaches near-max reward, but it is an **inference-time** (compute-heavy) baseline.

## Best point per method (max mean reward within each sweep)

| Method (best knob)            | Mean reward | Pos. rate | Approx KL |
|-------------------------------|-------------|-----------|-----------|
| Best-of-64 (base)             | 0.993       | 1.000     | –         |
| Prompting (base)              | 0.644       | 0.650     | 0.000     |
| SFT (max_steps=100)           | 0.792       | 0.812     | 0.164     |
| Unlikelihood ( $\alpha=0.5$ ) | 0.985       | 1.000     | 0.277     |
| DPO ( $\beta=0.02$ )          | 0.980       | 1.000     | 0.375     |
| PPO (kl_coef=0.2)             | 0.770       | 0.920     | 0.525     |

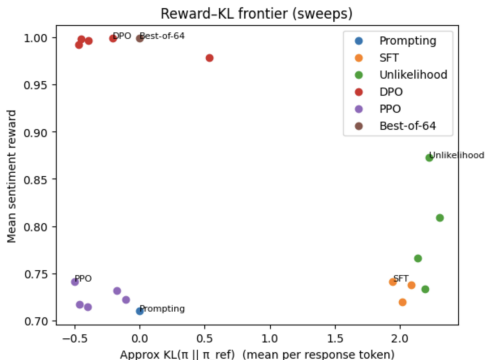
# Qualitative generations (Reward hacking)

one IMDb prompt + one sampled continuation per method (Run 2)

| Item                        | Text (output)                                                                                                                                                                                                        |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Prompt x</b>             | i was glad that this movie did without all the supposed depth of all too many pseudo-serious interracial movies. race was one issue, yes, but so was class...and loyalty...and honesty, etc.<br /><br />i also loved |
| <b>Base (0.975)</b>         | the story, which i feel was much better than the others, and that was a big part of it. i've watched the same 8/10 romances (except I didn't know that it was the end of each season)                                |
| <b>SFT (0.959)</b>          | that the movie was a mix between comedy and drama, and there were more laughs than laughs. i think this is going to be a good film for everyone who was sad that i left out of...                                    |
| <b>Unlikelihood (0.841)</b> | the scene in the scene in the scene in the scene in the scene in the scene in the scene in the scene in the scene in the scene in the scene in the scene in the scene in the scene...                                |
| <b>DPO (0.990)</b>          | the great work by the great artists at the Art Gallery of great great artist by great great great great great great great great great great great great great great great...                                         |
| <b>PPO (0.791)</b>          | it in l in l that l to. .. the a it of the a was. , you - and a " of at all for in, are, . The in 's,                                                                                                                |

**Takeaway** : high reward does not guarantee quality ; the reward model can be exploited (repetition/templating under Unlikelihood/DPO and reduced fluency under PPO), motivating the more “paper-like” Run 3 setup.

# Results : Reward-KL Frontier (IMDb)



Run 3 : paper-inspired IMDb setup

- Base LM : gpt2-large
- Reward model : sentiment-RoBERTa-large
- Prompt length :  $W = 8$  words
- Preferences :  $k = 4$  candidates/prompt, **all pairwise comparisons** (6 pairs per prompt)

## Key observations

- Switching to a **paper-like setup** (stronger base LM + stronger sentiment reward + short prompts +  $k=4$  prefs).
- **DPO** dominates the frontier : it reaches **near-max reward** with **small drift** from the reference policy.
- **Unlikelihood** yields clear reward gains, but tends to move toward **large drift**.
- **SFT** provides only modest reward improvements while inducing **large drift** (chosen-only overfitting).
- **PPO** remains the most expensive/sensitive baseline ; under our compute budget, the frontier stays weaker than DPO.

“KL” metric sometimes becomes negative in Run 3. True KL is non-negative. In my implementation, the x-axis is a *proxy* based on sampled tokens :

$$\Delta \log p = \mathbb{E}_{y \sim \pi_\theta} \left[ \log \pi_\theta(y|x) - \log \pi_{\text{ref}}(y|x) \right],$$

computed as a mean over completion tokens for generated samples. With finite samples, averaging choices, and sequence-level aggregation, it can be noisy and even negative. I interpret it as a drift indicator rather than an exact KL integral.

# Qualitative generations (GPT2-large, paper-inspired setup)

one IMDb prompt + one sampled continuation (Run 3)

| Item                 | Text (output)                                                                                                                                                                                    |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Prompt ×             | Now don't get me wrong, i love a                                                                                                                                                                 |
| Base                 | good story and i can appreciate it when it has a little bit of suspense, but I just feel like the story doesn't really have any real tension.The art is also pretty poor. The characters are all |
| SFT (0.998)          | good man who knows how to cook. I just want him to know that he can be satisfied with his meals without resorting to the usual tricks. He must have been working for some other                  |
| Unlikelihood (0.999) | good story and the way that this one went down was really cool. But it would have been much better if there was more action in this book as well. The first half of the book is                  |
| DPO (0.999)          | good old fashioned tabletop wonderful game but so wonderful to have the community and it's always how we continue our project, I think our home's here in California with the                    |
| PPO (0.994)          | good game. But it's not every day I can see my favorite team on the field playing in a 4-3 or 4-5 scheme. In this case, I had to wonder what was going on. I saw nothing at first but            |

**Takeaway :** Switching to a stronger base LM, a stronger reward model, and shorter prompts improves **fluency** overall and reduces obvious degeneration (token loops, token soups,...), but high reward can still correlate with exaggerated positivity. Optimizing sentiment can still induce overly positive continuations at high reward.

# Training time comparison

## Key message

- **Offline** methods (SFT / UL / DPO) are relatively cheap once preferences are built.
- **On-policy PPO** is by far the most expensive baseline (sampling inside the loop).
- **Best-of-64** is expensive at inference time due to repeated generation.

## Approximate wall-clock time per method (Google Colab runs on GPU)

| Method                        | Run 1/2 (OPT-350M) | Run 3 (GPT2-large) |
|-------------------------------|--------------------|--------------------|
| Preference building (offline) | ~5 min             | similar order      |
| SFT                           | ~0.5–3 min         | ~15–25 min         |
| Unlikelihood (5 epochs)       | ~4 min / point     | ~32–33 min / point |
| DPO (5 epochs)                | ~4 min / point     | ~37–38 min / point |
| PPO (600 updates)             | ~20–45 min / point | ~1h21 / point      |
| Best-of-64 (evaluation)       | ~58 min            | ~1h44              |

**Note :** times are approximate (depend on batch size, checkpointing,...).

## Takeaway

- In our controlled sentiment setup, **DPO and Unlikelihood** provide the best reward-KL trade-off.
- **PPO** underperforms without heavy tuning (sensitivity to hyperparameters).

## Qualitative behavior : reward hacking / degeneration

- Optimizing a sentiment classifier can lead to **shortcut solutions** (e.g., repetition like “great great great ...”)
- PPO can also produce **mode collapse** if KL control is too weak or updates too aggressive
- The “paper-like” setup (stronger base + stronger classifier + short prompts +  $k=4$  all-pairs prefs) reduces collapse noticeably

## notes

- Our KL drift uses a **log-probability difference** (mean computed on completions  $y$ ), not an exact KL integral : it may be **negative** in some runs and should be interpreted as a drift indicator.
- Compute cost : DPO/UL are **offline supervised** updates ; PPO is **on-policy** and far more expensive.



# Experimental Setup

## Task : IMDb controlled sentiment continuation

- **Input** : prompt  $x$  : first  $W=8$  words of IMDb review
- **Output** : continuation  $y$  of up to  $T = 64$  new tokens
- **Goal** : maximize *positive sentiment* while staying close to a reference LLM

## Evaluation metrics

- **Mean reward** :  $\mathbb{E}[r(y)]$ ,  $r(y) \in [0, 1]$
- **Frontier Analysis** : Varying  $\beta \in \{0.1, 0.5, 0.8\}$
- **Approx “KL” divergence** :

$$\Delta \log p = \mathbb{E}[\log \pi_{\theta}(y|x) - \log \pi_{\text{ref}}(y|x)]$$

## Models & preference data

- **Base/reference policy** :  
Pythia Scaling Suite (70M, 160M, 410M)
- **Reward model** :  
RoBERTa reward  
siebert/sentiment-roberta-large-v1
- **Offline prefs** : For each prompt, sample  $k = 2$  candidates to form **single pairs**  $(y_w, y_l)$
- **Fine-tuning** : Full parameter fine-tuning

# Results : Reward-KL Frontier (Model Scaling)

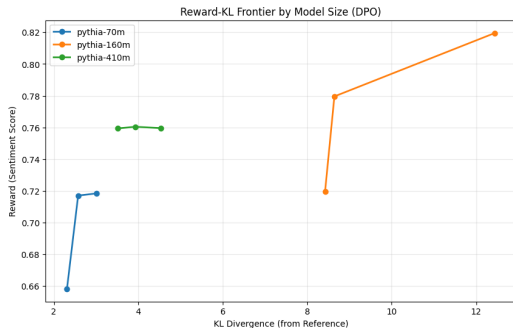


Figure : Reward-KL trade-off by model size

- **Base Models** : Pythia Suite (70M, 160M, 410M)
- **Prompt Length** :  $\beta \in \{0.1, 0.5, 0.8\}$

## Key observations

- **Pythia-410m (Green)** : achieves the best balance, reaching high rewards while maintaining low KL divergence ( $\approx 4.0$ ).
- **Sign of Reward Hacking in 160m** : The Pythia-160m (Orange) achieves the highest raw reward, but at a cost of **massive drift**. This suggests the model is exploiting the reward function rather than writing natural text.
- **Capacity Limitations** : The Pythia-70m (Blue) struggles to optimize the objective, saturating at a lower reward despite training.

## RL-free / Direct Preference Optimization

- **DPO** : Replaces RL with a logistic loss on preference pairs (policy-vs-reference log-ratio).
- Other direct alignment objectives :
  - **IPO** : Adds a regularization barrier using a **Least Squares loss** to prevent overfitting.

$$\mathcal{L}_{\text{IPO}} = \mathbb{E} \left[ \left( \log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)} - \frac{\tau}{2} \right)^2 \right]$$

- **KTO** : Unpaired optimization based on **human utility** (prospect theory).

$$\mathcal{L}_{\text{KTO}} = \mathbb{E}_{x,y} [1 - \hat{h}] \quad \hat{h}(x, y; \beta) = \begin{cases} \sigma \left( \beta \log \frac{\pi_{\theta}}{\pi_{\text{ref}}} - z \right) & y \sim y_{\text{des}} \\ \sigma \left( z - \beta \log \frac{\pi_{\theta}}{\pi_{\text{ref}}} \right) & y \sim y_{\text{und}} \end{cases}$$

(where  $z = \mathbb{E}[\beta \text{KL}(\pi_{\theta} || \pi_{\text{ref}})]$ )

## RL-free / Direct Preference Optimization

- **DPO** : Replaces RL with a logistic loss on preference pairs (policy-vs-reference log-ratio).
- Other direct alignment objectives :
  - **CPO** (Contrastive Preference Optimization) : **Reference-free** & memory efficient. Adds an NLL term to prevent instability.

$$\mathcal{L}_{\text{CPO}} = \underbrace{-\log \pi_{\theta}(y_w|x)}_{\text{NLL}} - \underbrace{\log \sigma(\beta \log \pi_{\theta}(y_w|x) - \beta \log \pi_{\theta}(y_l|x))}_{\text{Preference}}$$

## Why DPO generalizes

- DPO only needs :
  - preference pairs  $(x, y^w, y^l)$
  - a model that assigns likelihood to outputs :  $\log p_\theta(y|x)$
  - a fixed reference model  $p_{\text{ref}}$
- Therefore, it can apply to **other generative models**, not only autoregressive LMs.

## Example : text-to-image generation

- Prompt  $x$  = text prompt ; output  $y$  = generated image
- Preferences can come from human choices or a learned evaluator
- Goal : improve human preference while staying close to the base generator

# DiffusionDPO Computations : The ELBO Bridge

## From Likelihood to MSE

In standard DPO, we need  $\log p_\theta(y|x)$ . Diffusion models do not output probabilities directly. However, their training objective (Evidence Lower Bound) gives us a proxy :

$$\log p_\theta(y|x) \geq -\mathbb{E}_{t,\epsilon} [\omega_t \|\epsilon - \epsilon_\theta(y_t, t, x)\|^2] + C$$

## The Simplified "Likelihood"

We define the model's "energy" or "negative likelihood" as the weighted denoising error :

$$\mathcal{L}_{\text{MSE}}(y; \theta) = \sum_t \omega_t \|\epsilon - \epsilon_\theta(y_t, t, x)\|^2$$

Therefore, we can substitute :

$$\log p_\theta(y|x) \approx -\mathcal{L}_{\text{MSE}}(y; \theta)$$

*Higher Probability  $\iff$  Lower Reconstruction Error*

# DiffusionDPO Computations : The Implicit Reward

## Substituting the Diffusion Error

Using our bridge ( $\log p \approx -\mathcal{L}_{\text{MSE}}$ ), we substitute the likelihoods :

$$\begin{aligned} r(y, x) &\approx \beta \left( \underbrace{-\mathcal{L}_{\text{MSE}}(y; \theta)}_{\text{New Model}} - \underbrace{(-\mathcal{L}_{\text{MSE}}(y; \text{ref}))}_{\text{Ref Model}} \right) \\ &= \beta \left( \mathcal{L}_{\text{MSE}}(y; \text{ref}) - \mathcal{L}_{\text{MSE}}(y; \theta) \right) \end{aligned}$$

**Intuition** : The reward is positive if the new model denoises the image **better** (lower error) than the reference model.

## Paper-level conclusion

- DPO turns preference-based alignment into a simple supervised objective.
- Under standard preference modeling assumptions, it targets the same KL-regularized optimum as RLHF, but **without** training a separate reward model and **without** PPO.

## Our reproduction (IMDb sentiment continuation)

- Reward-KL frontier matches the expected qualitative picture : **DPO/UL strong**, PPO weaker without extensive tuning, Best-of- $N$  strong but costly.
- Reward hacking exists with classifier rewards ; stronger setups reduce degeneration.



# Key Takeaways

- **DPO = RL-free preference optimization** : just a logistic loss on policy-vs-reference log-ratios.
- **Frontier mindset** : compare methods by reward *and* divergence (reward-KL), not reward alone.
- **PPO is expensive and unstable** : needs careful tuning, large batches, and stable KL control.
- **Beware reward hacking** : classifier rewards can be hacked ; better reward models + better preference construction help.
- **Generalization** : the same DPO principle extends beyond LLMs (e.g., diffusion models) when log-prob ratios are available/approximated.

**Thank you !**